

Problem Statement:

When performing similarity searches in Milvus, the search results return a list of Hit objects. Each Hit object contains an `id` attribute, which represents the internal ID assigned by Milvus to the vector when it was inserted into the collection. This internal ID is crucial for identifying which vectors (in this case, journeys) are most similar to the query vector. The challenge arises in mapping these Milvus internal IDs (`hit.id`) back to the original user IDs (`user_id`) stored in a MySQL database.

Specific Issues Encountered:

- **IndexError on Accessing Journey Data:** You encountered an `IndexError` when attempting to use `hit.id` as an index to access elements in the `journey_data` list, which suggests a mismatch between the indices used in this list and the internal IDs assigned by Milvus.
- **Incorrect Assumption About ID Mapping:** The script incorrectly assumes that the internal IDs assigned by Milvus directly correlate with the indices of the `journey_data` list fetched from MySQL. However, Milvus' internal ID assignment does not necessarily follow the insertion order or align with the indices of a separate data structure maintained outside of Milvus.

Implications:

Without a reliable method to map Milvus internal IDs back to the original user IDs, it becomes challenging to accurately identify which users' journeys are most similar to the query journey. This mapping issue hinders the ability to present meaningful similarity search results that are actionable and understandable in terms of the original dataset.

Potential Solutions:

To resolve this mapping issue, consider the following approaches:

- **Maintain a Separate Mapping:** When inserting vectors into Milvus, keep track of the internal IDs returned by Milvus and create a mapping between these internal IDs and your original user IDs. This mapping can be stored in your MySQL database or another suitable data structure.
- **Use External IDs as Vector Fields:** Another approach involves storing the `user_id` as an additional field in each vector document within Milvus, assuming your version of Milvus supports such functionality. This way, you can retrieve the `user_id` directly from the search results without relying on indirect mapping.
- **Review Data Insertion Logic:** Ensure that the logic for inserting data into Milvus and the subsequent retrieval and mapping process aligns with how Milvus assigns and manages internal vector IDs. Adjust your application logic to accommodate Milvus' ID management practices.

Addressing this issue requires a careful review of how vector data is managed within Milvus and the implementation of a robust mapping strategy that accurately links Milvus search results with the original dataset.

Different Approach to Search

- **Integrated Indexing and Search:** Azure Cognitive Search provides a fully managed search service that integrates data ingestion (indexing) with search capabilities. When you index your data, you have control over the schema of your index, including defining fields that can store your original user IDs. This direct control over the schema allows you to include `user_id` as a field in your search index, ensuring it's always available and directly associated with each document (or vector) in your search results.
- **Search Capabilities:** Azure Cognitive Search supports various types of searches, including full-text search, filters, faceted search, and more. For similarity searches based on vector data, you would need to look into Azure's support for vector search or how you might implement similarity search using the available features. As of my last update, Azure introduced a preview of vector search capabilities, enabling similarity search over high-dimensional vectors. It's important to review the current offerings and limitations.

Implementation Considerations

- **Schema Design:** You have the flexibility to design your index schema in a way that includes all necessary fields to directly relate search results back to your original dataset. This includes the use of `user_id` as a field in the schema.
- **Querying and Results Handling:** When performing searches, Azure Cognitive Search returns documents that match your query criteria, including any fields you've included in the schema (such as `user_id`). This direct inclusion of `user_id` in search results inherently avoids the ID mapping issue encountered with Milvus.
- **Service Features and Updates:** Azure continuously updates its services, including Azure Cognitive Search. It's beneficial to review the latest features and documentation, especially regarding vector search capabilities and any features designed to support similarity searches.

Conclusion

While switching to Azure Cognitive Search can provide mechanisms that potentially avoid the ID mapping issue by leveraging schema design and integrated indexing/search capabilities, success depends on how you implement and utilise these features. It's crucial to thoroughly review Azure Cognitive Search's capabilities, especially around vector search and similarity search, to ensure it meets your specific application requirements and to understand any new challenges that might arise.